

```
>>> c = Context({"name": "Rumpelstiltskin"})
>>> t.render(c)
'Hello Rumpelstiltskin'
```

Render the template using the given context. The result replaces `{{ name }}` with Rumpelstiltskin.

A dictionary containing data for the context

The template language allows you to access the attributes on objects, the items in lists and tuples, and the values in dictionaries. All of these are done using a dot notation, even those parts that would require square brackets to access in Python. For example, `{{ person.name }}` works for both a person object and a person dictionary, accessing the name attribute or key, respectively. The third item in a list of fruit is accessed using `{{ fruit.2 }}`.

Dot notation also gets used for callables without arguments. For example, `{{ person.show_name }}` invokes the `person.show_name()` method and renders its results.

In addition to doing variable replacement with double braces, you can use template tags to control the content of your rendered result. The most common tags are conditional blocks and loops, but some are for straight replacement as well. The `{% lorem %}` tag gets rendered as *lorem ipsum* text, a chunk of Latin often used in the printing industry to indicate placeholder text. On your website, you might use the `{% lorem %}` tag when you're waiting on someone else to write some copy, dropping the tag in so that the page renders with some content taking up space on the page, instead of just leaving the area blank. This may not be the most popular tag, but it is a good example to help learn how tags work.

In the REPL, create a new `Template` object that uses the `{% lorem %}` tag. As your template doesn't have any data, use an empty dictionary to instantiate the `Context`. The following listing contains the code and the resulting Latin.

Listing 3.2 Using the `{% lorem %}` tag in the Django REPL

```
>>> t = Template("Cicero said: '{% lorem %}', amongst other things")
>>> t.render(Context({}))
"Cicero said: 'Lorem ipsum dolor sit amet, consectetur
adipiscing elit, sed do eiusmod tempor incididunt ut labore et
dolore magna aliqua. Ut enim ad minim veniam, quis nostrud
exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate
velit esse cillum dolore eu fugiat nulla pariatur. Excepteur
sint occaecat cupidatat non proident, sunt in culpa qui officia
deserunt mollit anim id est laborum.', amongst other things"
```

Like many tags, `{% lorem %}` takes optional arguments, allowing you to control how much text results. The first two arguments are a count and a method. The method can be `w`, `p`, or `b`, for *words*, *paragraphs*, or *blocks*. The value of the count tells you how many of the words, paragraphs, or blocks to include. Render another template in the REPL, this time telling `{% lorem %}` to display five words. The following listing shows the code and the resulting, much shorter, text.